

Chapter 13

Tools in LangChain

Giving LLMs hands, not just a brain — how LangChain lets a model search the web, run shell commands, call APIs, and execute custom Python functions as part of its reasoning loop.

WHAT THIS CHAPTER COVERS

- ▶ What a Tool is and why LLMs need them
- ▶ Built-in tools: DuckDuckGoSearchRun, ShellTool
- ▶ Creating custom tools with @tool decorator
- ▶ Creating custom tools with StructuredTool
- ▶ Creating custom tools with the BaseTool class
- ▶ Pydantic-based args_schema for structured input
- ▶ Grouping tools into a Toolkit
- ▶ How tool calling actually works end-to-end

Tools in LangChain

RECAP — CHAPTER 12

Chapter 12 built a complete RAG pipeline for a YouTube-chat assistant: indexing a transcript into a vector store, retrieving relevant chunks, augmenting a prompt with that context, and generating an answer — all wired together with LCEL (RunnableParallel, RunnableLambda, RunnablePassthrough). That pipeline let the LLM answer questions about existing content. This chapter introduces the missing piece for an LLM to *act* on the world instead of just reading from it: Tools.

1. What Is a Tool in LangChain?

Definition

A *Tool* is simply a Python function that has been packaged in a way an LLM can understand and decide to call — it has a **name**, a **description** (used by the LLM to decide when to use it), and a defined **input schema** (what arguments it expects and their types). In LangChain, every tool is ultimately an instance of the `BaseTool` class, whether it was created with a one-line decorator or built by hand.

Why It Exists

An LLM by itself can only generate text based on what it was trained on. It cannot browse the live web, run code, query a database, check today's weather, or perform exact arithmetic reliably. Tools solve this by giving the model a way to request that a specific, deterministic piece of code be executed on its behalf, with the result fed back into the conversation.

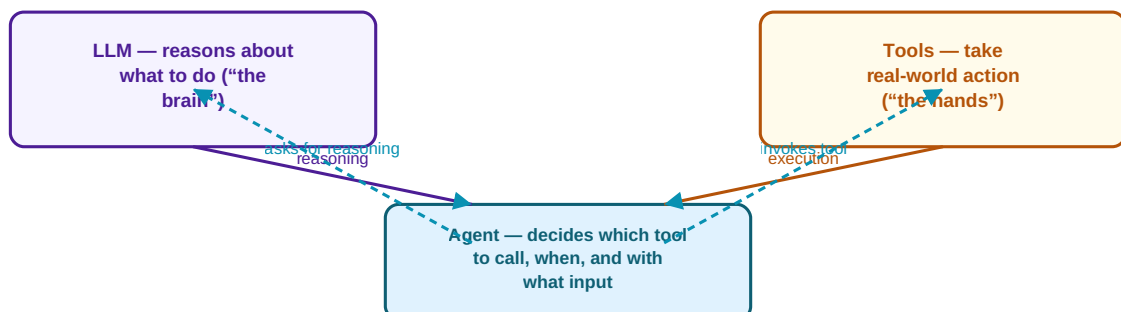


Figure 13.1 — The LLM reasons about what needs to happen; Tools execute real actions; an Agent sits between them, deciding which tool to call and with what input.

How It Works

1. The LLM is bound to a list of available tools, each described by its name, description, and argument schema.
2. When a user query needs external action or data, the LLM's output includes a structured "tool call" — a chosen tool name plus arguments matching its schema, instead of (or alongside) plain text.
3. LangChain's runtime intercepts this tool call, looks up the matching Tool object, and invokes it with `tool.invoke(arguments)`.
4. The tool executes normal Python code and returns a result, which is wrapped as a `ToolMessage` and appended back into the conversation.
5. The LLM reads the tool's result and produces the final natural-language answer.

IMPORTANT

A Tool on its own does nothing autonomous — it is a passive, callable object. It is the **LLM** that decides *when* to call a tool (based on the query and the tool's description), and it is the **Agent** runtime that actually executes that decision. Confusing "defining a tool" with "the model automatically using it" is a common early mistake.

🔄 Quick Revision — What Is a Tool

- A Tool is a Python function wrapped with a name, description, and input schema so an LLM can decide to call it.
- Every LangChain tool is ultimately a `BaseTool` instance under the hood.
- Tools give LLMs the ability to act (search, compute, call APIs, run commands) instead of only generating text.
- The LLM decides which tool to call and with what arguments; the runtime actually executes it.
- A tool's description is what the LLM reads to decide relevance — a vague description leads to the tool being picked incorrectly or never.

2. Built-in Tools

Definition

LangChain ships a large library of ready-made tools under `langchain_community.tools`, covering common needs like web search, running shell commands, querying Wikipedia, hitting public APIs, and more. These can be imported and used immediately — no custom function needs to be written.

Why It Exists

Many capabilities (web search, shell access, Wikipedia lookups) are needed across almost every agentic project. Rather than every engineer re-implementing a DuckDuckGo wrapper or a subprocess-running shell tool, LangChain's community package provides tested, standardized implementations that already follow the correct Tool interface.

1. DUCKDUCKGOSEARCHRUN — WEB SEARCH TOOL

A built-in tool that performs a live web search via DuckDuckGo and returns the results as plain text, with no API key required.

```
from langchain_community.tools import DuckDuckGoSearchRun

search_tool = DuckDuckGoSearchRun()

# .invoke() runs the actual search and returns text results
result = search_tool.invoke("SpiderMan: Brand New Day performance")

print(result)
```

Calling `.invoke()` on any LangChain tool — built-in or custom — is the standardized way to execute it directly, exactly the same call pattern an agent uses internally when it decides to invoke the tool.

2. SHELLTOOL — RUN SHELL COMMANDS

A built-in tool that executes a shell command on the host machine and returns its stdout output.

```
from langchain_community.tools import ShellTool

shell_tool = ShellTool()

# Runs "ls" in the shell and captures the output
result = shell_tool.invoke("ls")

print(result)
```

COMMON MISTAKE

Using `ShellTool` in a production agent without sandboxing or restricting which commands can run is a serious security risk — an LLM that can be tricked into generating a destructive shell command (e.g. via prompt injection from an untrusted document) will have that command actually executed on the host.

BEST PRACTICE

Treat any tool that touches the filesystem, network, or shell as a privileged capability. Restrict it to a sandboxed environment, log every invocation, and avoid giving an agent unrestricted `ShellTool` access in anything user-facing.

Quick Revision — Built-in Tools

- `langchain_community.tools` ships ready-made tools like `DuckDuckGoSearchRun` and `ShellTool`.
- `DuckDuckGoSearchRun` performs a live web search and needs no API key.
- `ShellTool` executes shell commands on the host machine and returns `stdout`.
- All tools, built-in or custom, are invoked the same way: `tool.invoke(input)`.
- Shell and filesystem-level tools carry real security risk and should be sandboxed.

3. Creating Custom Tools — The `@tool` Decorator

Definition

The `@tool` decorator is the fastest way to turn an ordinary, type-hinted Python function into a LangChain Tool. It reads the function's name, type hints, and docstring to automatically build the tool's name, argument schema, and description.

Why It Exists

Writing a full `BaseTool` subclass for every small function (like `add` or `multiply`) is unnecessary boilerplate. The decorator approach lets an engineer go from "plain function" to "agent-usable tool" in three small, incremental steps, keeping the code close to normal Python.

How It Works

1. **Step 1 — Write a plain function** with a clear docstring describing what it does.
2. **Step 2 — Add type hints** to every parameter and the return value, since LangChain uses these to build the tool's Pydantic input schema.
3. **Step 3 — Apply `@tool`** above the function definition. The function is now a Tool object, not a plain callable.

```

from langchain_community.tools import tool

# Step 1: plain function
def multiply(a, b):
    """Multiply two numbers"""
    return a * b

# Step 2: add type hints (LangChain uses these to build the input schema)
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

# Step 3: decorate – this is now a LangChain Tool object
@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

result = multiply.invoke({"a": 5, "b": 7})

print(multiply.name)           # "multiply"
print(multiply.description)    # "Multiply two numbers"
print(multiply.args)          # {'a': {...}, 'b': {...}}
print(multiply.args_schema.model_json_schema()) # full JSON schema

```

Once decorated, `multiply` is no longer called like a normal Python function (`multiply(5, 7)`); it must be invoked through the LangChain interface with a dictionary of named arguments: `multiply.invoke({"a": 5, "b": 7})`. The decorator also auto-generates a Pydantic input schema behind the scenes, accessible via `.args_schema.model_json_schema()`, which is exactly what gets sent to the LLM so it knows how to format a call to this tool.

COMMON MISTAKE

Forgetting the docstring, or writing a vague one like `"does stuff"`. The docstring is the tool's description, and the LLM relies entirely on it to decide whether this tool is relevant to a given query. A missing or unclear docstring will cause the LLM to either never call the tool or call it for the wrong reason.

INTERVIEW TIP

Be ready to explain *why* type hints are mandatory for `@tool`-decorated functions: LangChain introspects the function signature to build a Pydantic `args_schema` automatically, and without type hints it cannot infer the expected argument types for validation.

Quick Revision — @tool Decorator

- @tool converts a type-hinted, docstring-documented function into a LangChain Tool in three steps.
- The docstring becomes the tool's description — the single signal the LLM uses to decide relevance.
- Type hints are used by LangChain to auto-build a Pydantic args_schema for the tool.
- A decorated tool must be called via .invoke({...}) with named arguments, not called directly like a normal function.
- tool.name, tool.description, tool.args, and tool.args_schema.model_json_schema() expose the tool's full metadata.

4. Creating Custom Tools — StructuredTool

Definition

StructuredTool is a class that builds a Tool from an existing plain Python function via its .from_function() classmethod, letting the engineer explicitly pass the **name**, **description**, and a separate Pydantic **args_schema** instead of relying on decorator inference.

Why It Exists

The @tool decorator infers everything from the function signature and docstring, which is convenient but rigid. When a tool needs a richer input schema — field-level descriptions, validation constraints, or reused schema classes across multiple tools — StructuredTool gives explicit, separated control over each piece of metadata.

How It Works

1. Define a Pydantic BaseModel describing the expected input fields, each with a Field(description=...).
2. Write a plain function that performs the logic (it does *not* need to be decorated).
3. Call StructuredTool.from_function(), passing the function, an explicit name, description, and the args_schema class.

```

from langchain_community.tools import StructuredTool
from pydantic import BaseModel, Field

# 1. Explicit Pydantic input schema with per-field descriptions
class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number")
    b: int = Field(required=True, description="The second number")

# 2. Plain function – no decorator needed here
def multiplyfunc(a, b) -> int:
    return a * b

# 3. Wrap the function into a Tool, wiring up name/description/schema explicitly
multiply_tool = StructuredTool.from_function(
    func=multiplyfunc,
    name="multiply",
    description="Multiply two numbers",
    args_schema=MultiplyInput
)

result = multiply_tool.invoke({"a": 4, "b": 5})

print(multiply_tool.name)
print(multiply_tool.description)
print(multiply_tool.args)

```

BEST PRACTICE

Prefer `StructuredTool` over the `@tool` decorator once a tool's input has more than one or two simple fields, or when the same input schema needs to be reused across several tools — defining it once as a `BaseModel` keeps validation logic centralized and consistent.

Quick Revision — StructuredTool

- `StructuredTool.from_function()` builds a Tool from a plain (undecorated) function.
- Name, description, and `args_schema` are all passed explicitly rather than inferred.
- The input schema is a separate Pydantic `BaseModel` with per-field `Field(description=...)`.
- Useful when a tool's input is complex or the schema needs to be reused elsewhere.
- Still invoked the same way as any other tool: `tool.invoke({...})`.

5. Creating Custom Tools — The BaseTool Class

Definition

`BaseTool` is the abstract base class that every LangChain tool ultimately inherits from — including the ones produced by `@tool` and `StructuredTool` under the hood. Subclassing it directly gives full, unrestricted control over a tool's behavior.

Why It Exists

Some tools need custom logic beyond a single function call — conditional execution paths, internal state, custom error handling, or integration with an external SDK object. Subclassing `BaseTool` directly is the most flexible (and most verbose) way to define a tool, useful when the decorator and `StructuredTool` approaches are too constrained.

How It Works

1. Define a Pydantic `args_schema` exactly as with `StructuredTool`.
2. Subclass `BaseTool` and declare `name` and `description` as class attributes.
3. Set `args_schema: Type[BaseModel]` to point at the input schema class.
4. Implement the required `_run(self, ...)` method — this is where the actual tool logic lives.

```
from langchain_community.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type

# 1. Input schema (identical pattern to StructuredTool)
class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number")
    b: int = Field(required=True, description="The second number")

# 2 & 3. Subclass BaseTool, declare name/description/args_schema
class MultiplyTool(BaseTool):
    name: str = "multiply"
    description: str = "Multiply two numbers"
    args_schema: Type[BaseModel] = MultiplyInput

    # 4. _run() holds the actual tool logic
    def _run(self, a: int, b: int) -> int:
        return a * b

multiply_tool = MultiplyTool()

result = multiply_tool.invoke({"a": 3, "b": 3})

print(result)
```

@TOOL DECORATOR

```
@tool
def multiply(a: int,
            b: int) -> int:
    """Multiply two numbers"""
    return a * b

# fastest, least control
```

BASETOOL SUBCLASS

```
class MultiplyTool(BaseTool):
    name: str = "multiply"
    description: str = \
        "Multiply two numbers"
    args_schema = MultiplyInput

    def _run(self, a, b):
        return a * b

# most verbose, full control
```

1

@tool decorator

Fastest way — wrap a plain typed function with a docstring

Control level: Low

2

StructuredTool

from_function() — explicit name, description, args_schema

Control level: Medium

3

BaseTool subclass

Full control — custom class with _run() and args_schema

Control level: High

Figure 13.2 — The three ways to build a custom tool, ordered by how much control each gives you over the tool's behavior.

INTERVIEW TIP

A strong answer to "how many ways can you create a tool in LangChain?" names all three — `@tool`, `StructuredTool.from_function()`, and subclassing `BaseTool` — and explains the trade-off: decorator is fastest but least flexible, `BaseTool` is most verbose but gives full control (e.g. overriding async behavior, adding internal state, custom error handling).

Quick Revision — BaseTool Class

- `BaseTool` is the abstract base class underlying every LangChain tool, including decorator-built ones.
- Subclassing it directly requires declaring name, description, args_schema, and implementing `_run()`.
- This is the most verbose but most flexible way to build a custom tool.
- Useful for tools needing conditional logic, internal state, or custom error handling.
- All three creation methods — `@tool`, `StructuredTool`, `BaseTool` — produce an object invoked identically via `.invoke()`.

6. Toolkits — Grouping Related Tools

Definition

A *Toolkit* is a plain Python class that bundles a set of related tools behind a single `get_tools()` method, so an agent can be equipped with an entire family of capabilities in one line instead of importing and listing each tool individually.

Why It Exists

Real applications rarely need just one tool — a "math assistant" needs `add`, `multiply`, `subtract`, and more; a "file assistant" needs `read`, `write`, and `list-directory` tools. A Toolkit is an organizational convention for keeping such related tools together and exposing them as one importable, reusable unit.

How It Works

```
from langchain_community.tools import tool

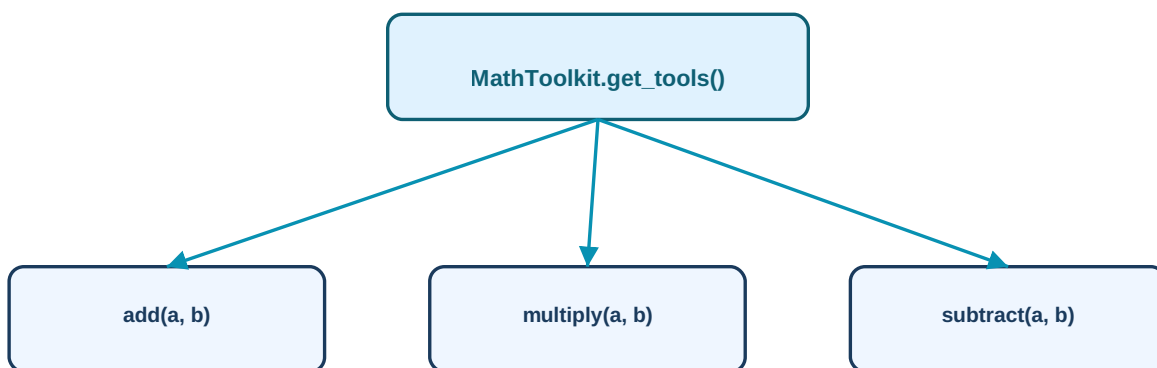
@tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

# A Toolkit is just a class that returns a list of tool objects
class MathToolkit:
    def get_tools(self):
        return [add, multiply]

toolkit = MathToolkit()
tools = toolkit.get_tools()

for tool in tools:
    print(tool.name, " => ", tool.description)
# add  =>  Add two numbers
# multiply  =>  Multiply two numbers
```



A Toolkit groups related `@tool`-decorated functions behind one `get_tools()` call, so an agent can be equipped with all of them in a single line.

Figure 13.3 — A Toolkit wraps several independently defined tools behind one `get_tools()` call.

BEST PRACTICE

Organize tools into toolkits along domain boundaries (math, file-system, database, search) rather than dumping every tool the application owns into one giant flat list — this keeps agent setup code readable and makes it easy to equip different agents with different capability sets.

Quick Revision — Toolkits

- A Toolkit is a plain class exposing a `get_tools()` method that returns a list of related Tool objects.
- It is purely an organizational pattern — there is no special LangChain base class required to build one.
- Toolkits let an agent be equipped with a whole family of capabilities in a single call.
- Group tools by domain (math, filesystem, search) for readable, reusable agent setup code.

7. How Tool Calling Actually Works End-to-End

Definition

Tool calling is the mechanism by which a tool-bound LLM's raw output is turned into an actual function execution and back into conversation context, without the developer manually parsing free-text output to figure out what the model "wants" to do.

How It Works

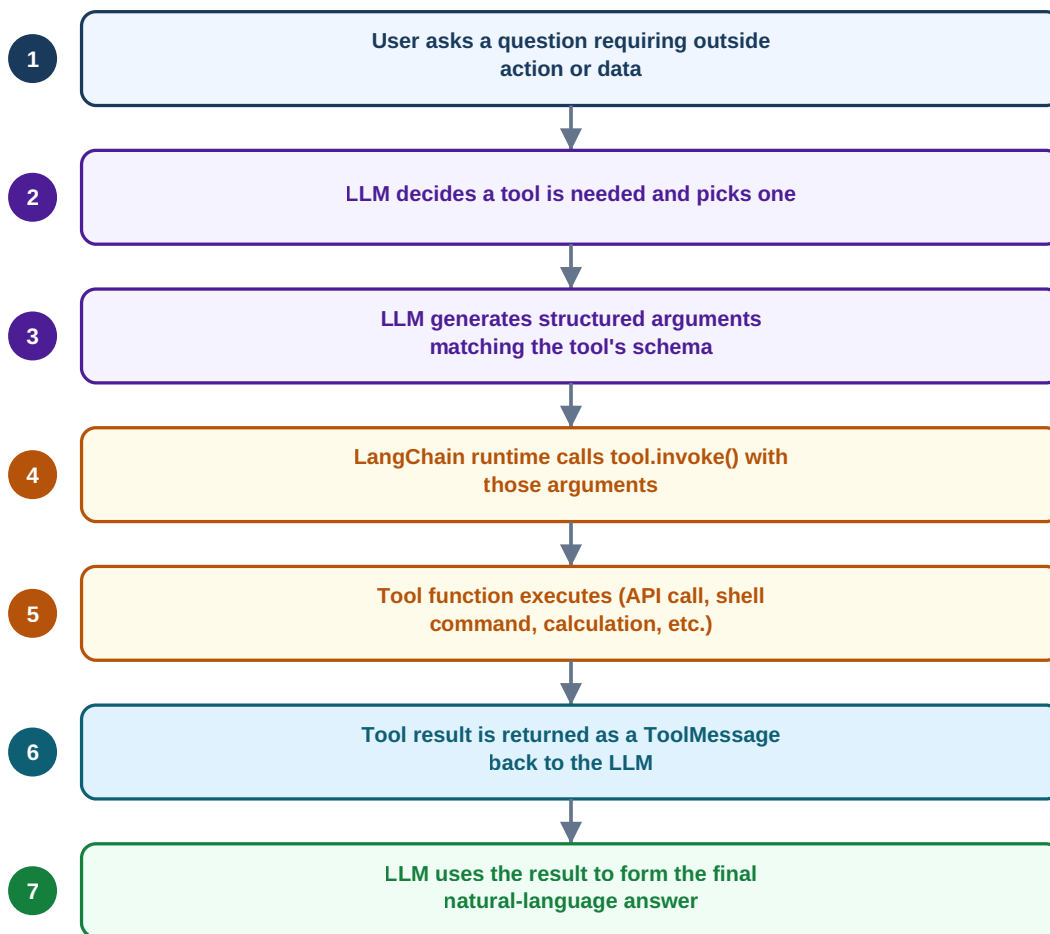


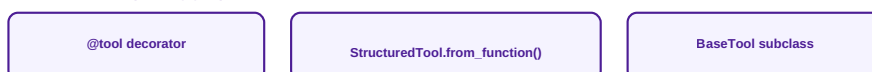
Figure 13.4 — The full lifecycle of a single tool call, from user question to final answer.

Comparison Point	Built-in Tools	Custom Tools
Source	<code>langchain_community.tools</code>	Defined by the engineer
Setup effort	Import and instantiate	Write function/class + schema
Examples	DuckDuckGoSearchRun, ShellTool	add, multiply, business-logic tools
Invocation	<code>tool.invoke(input)</code>	<code>tool.invoke(input)</code> — identical

BUILT-IN TOOLS (`langchain_community.tools`)



CUSTOM TOOLS (built by you)



Both kinds are plain LangChain Tool objects once created — an agent cannot tell the difference.

Figure 13.5 — Built-in and custom tools are interchangeable to an agent once created — both are plain Tool objects.

REAL WORLD INSIGHT

This tool-calling mechanism is the foundation that later chapters on Agents build directly on top of — an Agent is essentially a loop that repeatedly asks the LLM "do you want to call a tool, or are you ready to answer?" until the LLM decides it has enough information to respond.

COMMON MISTAKE

Assuming that simply importing or defining a tool makes an LLM automatically aware of it. The tool must be explicitly *bound* to the model (e.g. via `.bind_tools([...])` in an agent setup) before the LLM's output can include a call to it — a defined-but-unbound tool is invisible to the model.

Quick Revision — Tool Calling Lifecycle

- Tool calling turns an LLM's structured decision into an actual function execution and back.
- The LLM must be explicitly bound to its available tools before it can call any of them.
- The full loop: question -> LLM picks a tool + args -> runtime invokes it -> result returned as ToolMessage -> LLM forms final answer.
- Built-in and custom tools are invoked identically once created — an agent treats them the same way.
- This mechanism is the direct foundation for how Agents operate, covered in the next chapters.

K. Key Takeaways

- **Tools** are Python functions packaged with a name, description, and input schema so an LLM can decide to call them for real-world action or data.
- **Built-in tools** like `DuckDuckGoSearchRun` and `ShellTool` ship in `langchain_community.tools` and are ready to use without writing custom code.
- The **@tool decorator** is the fastest way to build a custom tool: write a typed function with a docstring, then decorate it.
- **StructuredTool.from_function()** gives explicit control over name, description, and a separate Pydantic `args_schema` for more complex inputs.
- **BaseTool subclassing** is the most flexible (and most verbose) approach, implementing custom logic inside a required `_run()` method.
- **Toolkits** are plain classes that bundle related tools behind a single `get_tools()` method for clean, reusable agent setup.
- **Tool calling** is a defined lifecycle: the LLM must be bound to its tools, decides when to call one, the runtime executes it, and the result flows back as a `ToolMessage`.
- All tools — built-in or custom, however they were created — are invoked identically via **`tool.invoke(input)`**, making them fully interchangeable to an agent.

R. Revision Sheet

Tool

A function wrapped with name, description, and input schema so an LLM can choose to call it as part of reasoning.

Built-in Tools

Ready-made tools in `langchain_community.tools`, e.g. `DuckDuckGoSearchRun` (web search) and `ShellTool` (shell commands).

@tool Decorator

Turns a type-hinted, docstring-documented function into a Tool in one line — fastest method, least control.

StructuredTool

Built via `.from_function()`, with explicit name, description, and a separate Pydantic `args_schema` for complex inputs.

BaseTool

The abstract base class every tool inherits from; subclassing it directly gives full control via a custom `_run()` method.

Toolkit

A plain class exposing `get_tools()`, bundling related tools so an agent can be equipped with all of them at once.

Tool Calling Lifecycle

Bind tools to LLM -> LLM picks tool + args -> runtime invokes it -> result returned as `ToolMessage` -> LLM forms final answer.

.invoke()

The universal call pattern for any LangChain tool, taking a dictionary of named arguments matching its schema.

I. Interview Questions

- Q1.** What is a Tool in LangChain, and why can't an LLM perform these actions on its own?
- Q2.** What are the three main ways to create a custom tool in LangChain, and how do they differ in control versus effort?
- Q3.** Why are type hints mandatory when using the `@tool` decorator?
- Q4.** What role does a tool's docstring play, and what happens if it is vague or missing?
- Q5.** How does `StructuredTool` differ from the `@tool` decorator in how the input schema is defined?
- Q6.** What method must be implemented when subclassing `BaseTool` directly, and what goes inside it?
- Q7.** What is a Toolkit, and why would you group tools into one instead of listing them individually?
- Q8.** Walk through what happens, step by step, from a user's question to a tool's result being used in the final answer.
- Q9.** Why must a tool be explicitly bound to an LLM before the model can call it?
- Q10.** What security risks are introduced by tools like `ShellTool`, and how would you mitigate them in production?

F. Further Reading

- LangChain official documentation — "Tools" concept guide (python.langchain.com)
- LangChain official documentation — `langchain_community.tools` reference (DuckDuckGoSearchRun, ShellTool, and other community tools)
- Pydantic documentation — `BaseModel` and `Field`, used for every tool's `args_schema`
- LangChain documentation — Agents and tool-calling agent types (the direct next step after this chapter)